2核2G云服务器 95元/年起 立即购买 >

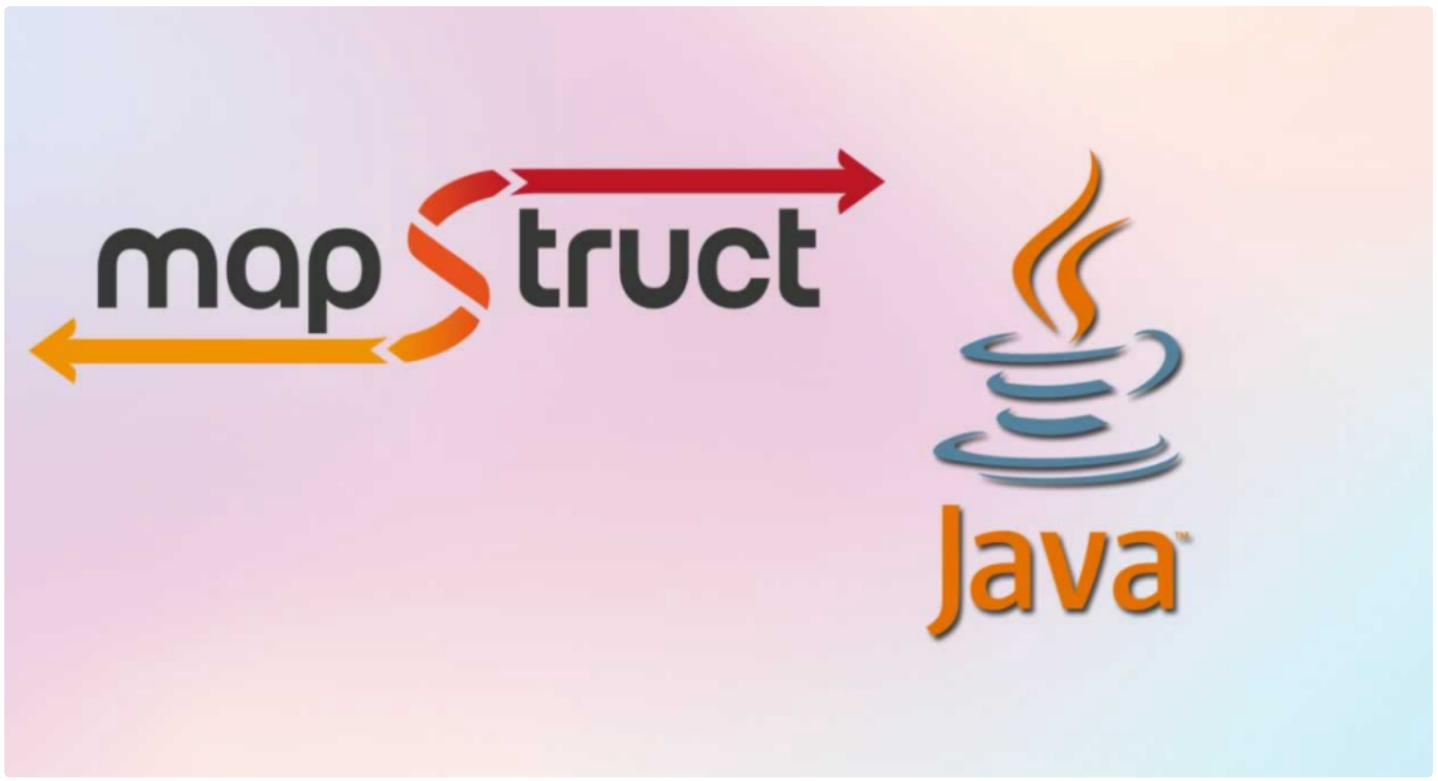
文档 意见反馈 控制台

首页 学习 活动 专区 更多 文章/答案/技术大牛 发布 登录/注册



# 实体映射最强工具类：MapStruct 真香！

发布于 2022-05-09 12:42:12 7.2K 1 举报




来源：blog.csdn.net/qq122516902

1.MapStruct是用来做什么的?
2.使用MapStruct解决上述问题
3.添加默认方法
4.可以使用abstract class来代替接口

5.可以使用多个参数
5.直接使用参数作为属性值
6.更新对象属性
7.没有getter/setter也能赋值
8.使用Spring依赖注入
9.自定义类型转换

首先来了解一下DTO，DTO简单的理解就是做数据传输对象的，类似于VO，但是VO用于传输到前端。（~~）

## 1.MapStruct是用来做什么的?

现在有这么个场景，从数据库查询出来了一个user对象（包含id，用户名，密码，手机号，邮箱，角色这些字段）和一个对应的角色对象role（包含id，角色名，角色描述这些字段），现在在controller需要用到user对象的id，用户名，和角色对象的角色名三个属性。

一种方式是直接把两个对象传递到controller层，但是这样会多出很多没用的属性。更通用的方式是需要用到的属性封装成一个类(DTO)，通过传输这个类的实例来完成数据传输。

User.java

```
@AllArgsConstructor
@Data
public class User {
    private Long id;
    private String username;
    private String password;
    private String phoneNum;
    private String email;
    private Role role;
}
```

java思维导图

513 文章 566.2K 累计阅读量 1.7K 获赞

关注 前往专栏

加入 开发者社群

领取 专属20元代金券
Get大咖技术交流圈

目录

1.MapStruct是用来做什么的?
2.使用MapStruct解决上述问题
3.添加默认方法
4. 可以使用abstract class来代替接口
5.可以使用多个参数
5.直接使用参数作为属性值
6.更新对象属性
7.没有getter/setter也能赋值
8.使用Spring依赖注入

广告
腾讯云
COS 1元礼包
提供 存储+处理 服务
立即抢购

Role.java

```
@AllArgsConstructor
@Data
public class Role {
    private Long id;
    private String roleName;
    private String description;
}
```

UserRoleDto.java，这个类就是封装的类

```
@Data
public class UserRoleDto {
    /**
     * 用户id
     */
    private Long userId;
    /**
     * 用户名
     */
    private String name;
    /**
     * 角色名
     */
    private String roleName;
}
```

测试类，模拟将user对象转换成UserRoleDto对象

```
public class MainTest {
    User user = null;

    /**
     * 模拟从数据库中查出user对象
     */
    @Before
    public void before() {
       Role role  = new Role(2L, "administrator", "超级管理员");
       user  = new User(1L, "zhangsan", "12345", "17677778888", "123@qq.com", role);
    }

    /**
     * 模拟把user对象转换成UserRoleDto对象
     */
    @Test
    public void test1() {
        UserRoleDto userRoleDto = new UserRoleDto();
        userRoleDto.setUserId(user.getId());
        userRoleDto.setName(user.getUsername());
        userRoleDto.setRoleName(user.getRole().getRoleName());
        System.out.println(userRoleDto);
    }
}
```

从上面代码可以看出，通过getter、setter的方式把一个对象属性值复制到另一个对象中去还是很麻烦的，尤其是当属性过多的时候。而MapStruct就是用于解决这种问题的。

### 2.使用MapStruct解决上述问题

这里我们沿用User.java、Role.java、UserRoleDto.java。

新建一个UserRoleMapper.java，这个来用来定义User.java、Role.java和UserRoleDto.java之间属性对应规则：

UserRoleMapper.java

```
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import org.mapstruct.Mappings;
import org.mapstruct.factory.Mappers;

/**
 * @Mapper 定义这是一个MapStruct对象属性转换接口，在这个类里面规定转换规则
 *          在项目构建时，会自动生成改接口的实现类，这个实现类将实现对象属性值复制
 */
@Mapper
public interface UserRoleMapper {
```

```
    /**
     * 获取该类自动生成的实现类的实例
     * 接口中的属性都是 public static final 的 方法都是public abstract的
     */
    UserRoleMapper INSTANCES = Mappers.getMapper(UserRoleMapper.class);

    /**
     * 这个方法就是用于实现对象属性复制的方法
     *
     * @Mapping 用来定义属性复制规则 source 指定源对象属性 target指定目标对象属性
     *
     * @param user 这个参数就是源对象，也就是需要被复制的对象
     * @return 返回的是目标对象，就是最终的结果对象
     */
    @Mappings({
            @Mapping(source = "id", target = "userId"),
            @Mapping(source = "username", target = "name"),
            @Mapping(source = "role.roleName", target = "roleName")
    })
    UserRoleDto toUserRoleDto(User user);

}
```

在测试类中测试：

通过上面的例子可以看出，使用MapStruct方便许多。

### 3.添加默认方法

添加默认方法是为了这个类（接口）不只是为了做数据转换用的，也可以做一些其他的事。

```
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import org.mapstruct.Mappings;
import org.mapstruct.factory.Mappers;

/**
 * @Mapper 定义这是一个MapStruct对象属性转换接口，在这个类里面规定转换规则
 *          在项目构建时，会自动生成改接口的实现类，这个实现类将实现对象属性值复制
 */
@Mapper
public interface UserRoleMapper {

    /**
     * 获取该类自动生成的实现类的实例
     * 接口中的属性都是 public static final 的 方法都是public abstract的
     */
    UserRoleMapper INSTANCES = Mappers.getMapper(UserRoleMapper.class);

    /**
     * 这个方法就是用于实现对象属性复制的方法
     *
     * @Mapping 用来定义属性复制规则 source 指定源对象属性 target指定目标对象属性
     *
     * @param user 这个参数就是源对象，也就是需要被复制的对象
     * @return 返回的是目标对象，就是最终的结果对象
     */
    @Mappings({
            @Mapping(source = "id", target = "userId"),
            @Mapping(source = "username", target = "name"),
            @Mapping(source = "role.roleName", target = "roleName")
    })
    UserRoleDto toUserRoleDto(User user);

    /**
     * 提供默认方法，方法自己定义，这个方法是我随便写的，不是要按照这个格式来的
     * @return
     */
    default UserRoleDto defaultConvert() {
        UserRoleDto userRoleDto = new UserRoleDto();
        userRoleDto.setUserId(0L);
        userRoleDto.setName("None");
        userRoleDto.setRoleName("None");
        return userRoleDto;
    }

```

```
}
```

测试代码：

```
@Test
public void test3() {
    UserRoleMapper userRoleMapperInstances = UserRoleMapper.INSTANCES;
    UserRoleDto userRoleDto = userRoleMapperInstances.defaultConvert();
    System.out.println(userRoleDto);
}
```

### 4. 可以使用abstract class来代替接口

mapper可以用接口来实现，也可以完全由抽象来完全代替

```
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import org.mapstruct.Mappings;
import org.mapstruct.factory.Mappers;

/**
 * @Mapper 定义这是一个MapStruct对象属性转换接口，在这个类里面规定转换规则
 *         在项目构建时，会自动生成改接口的实现类，这个实现类将实现对象属性值复制
 */
@Mapper
public abstract class UserRoleMapper {

    /**
     * 获取该类自动生成的实现类的实例
     * 接口中的属性都是 public static final 的 方法都是public abstract的
     */
    public static final UserRoleMapper INSTANCES = Mappers.getMapper(UserRoleMapper.class

    /**
     * 这个方法就是用于实现对象属性复制的方法
     *
     * @Mapping 用来定义属性复制规则 source 指定源对象属性 target指定目标对象属性
     *
     * @param user 这个参数就是源对象，也就是需要被复制的对象
     * @return 返回的是目标对象，就是最终的结果对象
     */
    @Mappings({
            @Mapping(source = "id", target = "userId"),
            @Mapping(source = "username", target = "name"),
            @Mapping(source = "role.roleName", target = "roleName")
    })
    public abstract UserRoleDto toUserRoleDto(User user);

    /**
     * 提供默认方法，方法自己定义，这个方法是我随便写的，不是要按照这个格式来的
     * @return
     */
    UserRoleDto defaultConvert() {
        UserRoleDto userRoleDto = new UserRoleDto();
        userRoleDto.setUserId(0L);
        userRoleDto.setName("None");
        userRoleDto.setRoleName("None");
        return userRoleDto;
    }

}
```

### 5.可以使用多个参数

可以绑定多个对象的属性值到目标对象中：

```
package com.mapstruct.demo;

import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import org.mapstruct.Mappings;
import org.mapstruct.factory.Mappers;

/**
 * @Mapper 定义这是一个MapStruct对象属性转换接口，在这个类里面规定转换规则
 *         在项目构建时，会自动生成改接口的实现类，这个实现类将实现对象属性值复制
```

```
 */
@Mapper
public interface UserRoleMapper {

    /**
     * 获取该类自动生成的实现类的实例
     * 接口中的属性都是 public static final 的 方法都是public abstract的
     */
    UserRoleMapper INSTANCES = Mappers.getMapper(UserRoleMapper.class);

    /**
     * 这个方法就是用于实现对象属性复制的方法
     *
     * @Mapping 用来定义属性复制规则 source 指定源对象属性 target指定目标对象属性
     *
     * @param user 这个参数就是源对象，也就是需要被复制的对象
     * @return 返回的是目标对象，就是最终的结果对象
     */
    @Mappings({
            @Mapping(source = "id", target = "userId"),
            @Mapping(source = "username", target = "name"),
            @Mapping(source = "role.roleName", target = "roleName")
    })
    UserRoleDto toUserRoleDto(User user);

    /**
     * 多个参数中的值绑定
     * @param user 源1
     * @param role 源2
     * @return 从源1、2中提取出的结果
     */
    @Mappings({
            @Mapping(source = "user.id", target = "userId"), // 把user中的id绑定到目标对象的u
            @Mapping(source = "user.username", target = "name"), // 把user中的username绑定到
            @Mapping(source = "role.roleName", target = "roleName") // 把role对象的roleNam
    })
    UserRoleDto toUserRoleDto(User user, Role role);
```

对比两个方法~

**5.直接使用参数作为属性值**

```
package com.mapstruct.demo;

import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import org.mapstruct.Mappings;
import org.mapstruct.factory.Mappers;

/**
 * @Mapper 定义这是一个MapStruct对象属性转换接口，在这个类里面规定转换规则
 *          在项目构建时，会自动生成改接口的实现类，这个实现类将实现对象属性值复制
 */
@Mapper
public interface UserRoleMapper {

    /**
     * 获取该类自动生成的实现类的实例
     * 接口中的属性都是 public static final 的 方法都是public abstract的
     */
    UserRoleMapper INSTANCES = Mappers.getMapper(UserRoleMapper.class);

    /**
     * 直接使用参数作为值
     * @param user
     * @param myRoleName
     * @return
     */
    @Mappings({
            @Mapping(source = "user.id", target = "userId"), // 把user中的id绑定到目标对象的u
            @Mapping(source = "user.username", target = "name"), // 把user中的username绑定到
            @Mapping(source = "myRoleName", target = "roleName") // 把role对象的roleName属性
    })
    UserRoleDto useParameter(User user, String myRoleName);

}
```

测试类：

```
public class Test1 {
    Role role = null;
    User user = null;

    @Before
    public void before() {
        role = new Role(2L, "administrator", "超级管理员");
        user = new User(1L, "zhangsan", "12345", "17677778888", "123@qq.com", role);
    }
    @Test
    public void test1() {
        UserRoleMapper instances = UserRoleMapper.INSTANCES;
        UserRoleDto userRoleDto = instances.useParameter(user, "myUserRole");
        System.out.println(userRoleDto);
    }
}
```

### 6.更新对象属性

在之前的例子中 `UserRoleDto useParameter(User user, String myRoleName);` 都是通过类似上面的方法来生成一个对象。而MapStruct提供了另外一种方式来更新一个对象中的属性。@MappingTarget

```
public interface UserRoleMapper1 {

    UserRoleMapper1 INSTANCES = Mappers.getMapper(UserRoleMapper1.class);

    @Mappings({
            @Mapping(source = "userId", target = "id"),
            @Mapping(source = "name", target = "username"),
            @Mapping(source = "roleName", target = "role.roleName")
    })
    void updateDto(UserRoleDto userRoleDto, @MappingTarget User user);


    @Mappings({
            @Mapping(source = "id", target = "userId"),
            @Mapping(source = "username", target = "name"),
            @Mapping(source = "role.roleName", target = "roleName")
    })
    void update(User user, @MappingTarget UserRoleDto userRoleDto);

}
```

通过@MappingTarget来指定目标类是谁（谁的属性需要被更新）。@Mapping还是用来定义属性对应规则。

以此为例说明：

```
@Mappings({
            @Mapping(source = "id", target = "userId"),
            @Mapping(source = "username", target = "name"),
            @Mapping(source = "role.roleName", target = "roleName")
    })
    void update(User user, @MappingTarget UserRoleDto userRoleDto);
```

`@MappingTarget` 标注的类UserRoleDto 为目标类，user类为源类，调用此方法，会把源类中的属性更新到目标类中。更新规则还是由 `@Mapping` 指定。

### 7.没有getter/setter也能赋值

对于没有getter/setter的属性也能实现赋值操作

```
public class Customer {

    private Long id;
    private String name;

    //getters and setter omitted for brevity
}

public class CustomerDto {

    public Long id;
    public String customerName;
}
```

```
@Mapper
public interface CustomerMapper {

    CustomerMapper INSTANCE = Mappers.getMapper( CustomerMapper.class );

    @Mapping(source = "customerName", target = "name")
    Customer toCustomer(CustomerDto customerDto);

    @InheritInverseConfiguration
    CustomerDto fromCustomer(Customer customer);
}
```

`@Mapping(source = “customerName”, target = “name”)` 不是用来指定属性映射的，如果两个对象的属性名相同是可以省略@Mapping的。

MapStruct生成的实现类：

```
@Generated(
    value = "org.mapstruct.ap.MappingProcessor",
    date = "2019-02-14T15:41:21+0800",
    comments = "version: 1.3.0.Final, compiler: javac, environment: Java 1.8.0_181 (Oracl
)
public class CustomerMapperImpl implements CustomerMapper {

    @Override
    public Customer toCustomer(CustomerDto customerDto) {
        if ( customerDto == null ) {
            return null;
        }

        Customer customer = new Customer();

        customer.setName( customerDto.customerName );
        customer.setId( customerDto.id );

        return customer;
    }

    @Override
    public CustomerDto toCustomerDto(Customer customer) {
        if ( customer == null ) {
            return null;
        }

        CustomerDto customerDto = new CustomerDto();

        customerDto.customerName = customer.getName();
        customerDto.id = customer.getId();

        return customerDto;
    }
}
```

`@InheritInverseConfiguration` 在这里的作用就是实现 `customerDto.customerName = customer.getName();` 功能的。如果没有这个注解，toCustomerDto这个方法则不会有customerName 和name两个属性的对应关系的。

### 8.使用Spring依赖注入

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Customer {
    private Long id;
    private String name;
}

@Data
public class CustomerDto {
    private Long id;
    private String customerName;
}

// 这里主要是这个componentModel 属性，它的值就是当前要使用的依赖注入的环境
@Mapper(componentModel = "spring")
```

```
public interface CustomerMapper {

    @Mapping(source = "name", target = "customerName")
    CustomerDto toCustomerDto(Customer customer);
}
```

`@Mapper(componentModel = “spring”)` ，表示把当前Mapper类纳入spring容器。可以在其它类中直接注入了：

```
@SpringBootApplication
@RestController
public class DemoMapstructApplication {

 // 注入Mapper
    @Autowired
    private CustomerMapper mapper;

    public static void main(String[] args) {
        SpringApplication.run(DemoMapstructApplication.class, args);
    }

    @GetMapping("/test")
    public String test() {
        Customer customer = new Customer(1L, "zhangsan");
        CustomerDto customerDto = mapper.toCustomerDto(customer);
        return customerDto.toString();
    }

}
```

看一下由mapstruct自动生成的类文件，会发现标记了 `@Component` 注解。

```
@Generated(
    value = "org.mapstruct.ap.MappingProcessor",
    date = "2019-02-14T15:54:17+0800",
    comments = "version: 1.3.0.Final, compiler: javac, environment: Java 1.8.0_181 (Oracl
)
@Component
public class CustomerMapperImpl implements CustomerMapper {

    @Override
    public CustomerDto toCustomerDto(Customer customer) {
        if ( customer == null ) {
            return null;
        }

        CustomerDto customerDto = new CustomerDto();

        customerDto.setCustomerName( customer.getName() );
        customerDto.setId( customer.getId() );

        return customerDto;
    }
}
```

### 9.自定义类型转换

有时候，在对象转换的时候可能会出现这样一个问题，就是源对象中的类型是Boolean类型，而目标对象类型是String类型，这种情况可以通过 `@Mapper` 的uses属性来实现：

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Customer {
    private Long id;
    private String name;
    private Boolean isDisable;
}

@Data
public class CustomerDto {
    private Long id;
    private String customerName;
    private String disable;
}
```

定义转换规则的类：

```
public class BooleanStrFormat {
    public String toStr(Boolean isDisable) {
        if (isDisable) {
            return "Y";
        } else {
            return "N";
        }
    }
    public Boolean toBoolean(String str) {
        if (str.equals("Y")) {
            return true;
        } else {
            return false;
        }
    }
}
```

定义Mapper，  `@Mapper( uses = { BooleanStrFormat.class})`  ，注意，这里的users属性用于引用之前定义的转换规则的类：

```
@Mapper( uses = { BooleanStrFormat.class})
public interface CustomerMapper {

    CustomerMapper INSTANCES = Mappers.getMapper(CustomerMapper.class);

    @Mappings({
            @Mapping(source = "name", target = "customerName"),
            @Mapping(source = "isDisable", target = "disable")
    })
    CustomerDto toCustomerDto(Customer customer);
}
```

这样子，Customer类中的isDisable属性的true就会转变成CustomerDto中的disable属性的yes。

MapStruct自动生成的类中的代码：

```
@Generated(
    value = "org.mapstruct.ap.MappingProcessor",
    date = "2019-02-14T16:49:18+0800",
    comments = "version: 1.3.0.Final, compiler: javac, environment: Java 1.8.0_181 (Oracl
)
public class CustomerMapperImpl implements CustomerMapper {

 // 引用 uses 中指定的类
    private final BooleanStrFormat booleanStrFormat = new BooleanStrFormat();

    @Override
    public CustomerDto toCustomerDto(Customer customer) {
        if ( customer == null ) {
            return null;
        }

        CustomerDto customerDto = new CustomerDto();
  // 转换方式的使用
        customerDto.setDisable( booleanStrFormat.toStr( customer.getIsDisable() ) );
        customerDto.setCustomerName( customer.getName() );
        customerDto.setId( customer.getId() );

        return customerDto;
    }
}
```

要注意的是，如果使用了例如像spring这样的环境，Mapper引入uses类实例的方式将是自动注入，那么这个类也应该纳入Spring容器：

CustomerMapper.java指定使用spring

```
@Mapper(componentModel = "spring", uses = { BooleanStrFormat.class})
public interface CustomerMapper {

    CustomerMapper INSTANCES = Mappers.getMapper(CustomerMapper.class);

    @Mappings({
            @Mapping(source = "name", target = "customerName"),
            @Mapping(source = "isDisable", target = "disable")
    })
```

```
    CustomerDto toCustomerDto(Customer customer);
}
```

转换类要加入Spring容器：

```
@Component
public class BooleanStrFormat {
    public String toStr(Boolean isDisable) {
        if (isDisable) {
            return "Y";
        } else {
            return "N";
        }
    }
    public Boolean toBoolean(String str) {
        if (str.equals("Y")) {
            return true;
        } else {
            return false;
        }
    }
}
```

MapStruct自动生成的类：

```
@Generated(
    value = "org.mapstruct.ap.MappingProcessor",
    date = "2019-02-14T16:55:35+0800",
    comments = "version: 1.3.0.Final, compiler: javac, environment: Java 1.8.0_181 (Oracl
)
@Component
public class CustomerMapperImpl implements CustomerMapper {

 // 使用自动注入的方式引入
    @Autowired
    private BooleanStrFormat booleanStrFormat;

    @Override
    public CustomerDto toCustomerDto(Customer customer) {
        if ( customer == null ) {
            return null;
        }

        CustomerDto customerDto = new CustomerDto();

        customerDto.setDisable( booleanStrFormat.toStr( customer.getIsDisable() ) );
        customerDto.setCustomerName( customer.getName() );
        customerDto.setId( customer.getId() );

        return customerDto;
    }
}
```

```
推荐阅读：看看人家 SpringBoot + vue后台管理系统，多么优雅...
```

**本文参与 腾讯云自媒体分享计划，分享自微信公众号。**

原始发表：2022-05-06，如有侵权请联系 cloudcommunity@tencent.com 删除

java 网络安全 spring

## 评论

登录 后参与评论

## 1 条评论

热度 最新

用户10096084 2022-10-09

太强了，这个工具类真好用啊

回复 点赞 举报

## 推荐

### 写一个ORM框架的第一步（Apache Commons DbUtils）

apache

新一次的内部提升开始了，如果您想写一个框架从Apache Commons DbUtils开始学习是一种不错的选择，我们先学习应用这个小“框架”再把源代码理解，然后写一个属于自己的ORM框架不是梦。 一、




张果 2018-01-04 854 0

### 初识Hibernate之环境搭建

nat java 数据库 jdbc

相信所有做后端的程序员同行们，没有不知道Hibernate大名的。这是一个经典的轻量级Java EE持久层的解决方案，它使得我们程序员能以面向对象的思维操作传统的关系型数据库，这也是其存在的最

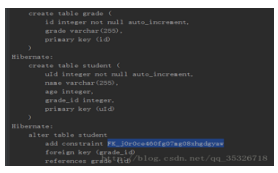


Single 2018-01-04 538 0

### 初识Hibernate之关联映射（一）

其他

上篇文章我们对持久化对象进行的学习，了解了它的三种不同的状态并通过它完成对数据库的映射操作。但这都是基于单张表的操作，如果两张或者两张以上的表之间存在某种关联，我们又该如何利用

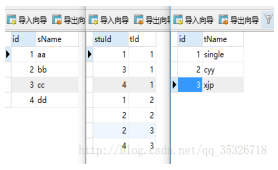


Single 2018-01-04 804 0

### 初识Hibernate之关联映射（二）

其他

上篇我们介绍了关联映射的几种形式，有单向多对一，单向一对多，还有双向一对多。本篇接着介绍有关关联映射的其他几种映射方式，主要有以下几种： 基于外键的单向一对一关联映射 基于主键的

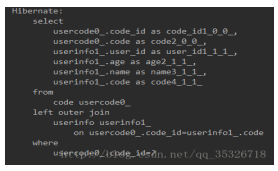



Single 2018-01-04 681 0

### Hibernate框架学习之注解配置关系映射

其他

上篇文章我们通过注解对映射了单个实体类，但是具体项目中往往实体类之间又是相互关联的，本篇文章就是从实体类之间存在的不同关联角度，具体学习下如何映射他们之间的关联，主要涉及内容如

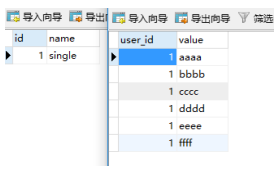


Single 2018-01-04 1.2K 0

### Hibernate框架学习之注解映射实体类

其他

前面的相关文章中，我们已经介绍了使用XML配置文件映射实体类及其各种类型的属性的相关知识。然而不论是时代的潮流还是臃肿繁杂的配置代码告诉我们，注解配置才是更人性化的设计，于是学

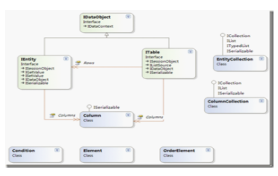



Single 2018-01-04 1.3K 0

### 基于DotNet构件技术的企业级敏捷软件开发平台 – AgileEAS.NET平台开发指南 – 数据层开发

.net

对象关系映射 AgileEAS.NETORM并没有采用如NHibernate中映射文件的文件的模式，而是采用了直接硬编码的模式实现，ORM体系设计采用了属性/列>数据对象>数据集合(表)的结构： ？



魏琼东 2018-01-12 1K 0

### eclipse从数据库逆向生成Hibernate实体类

其他

做项目必然要先进行数据库表设计，然后根据数据库设计建立实体类(VO)，这是理所当然的，但是到公司里做项目后，让我认识到，没有说既进行完数据库设计后还要再“自己”建立一变VO。意思是，在项目设计时，要么根据需求分析建立实体类，由正向

似水的流年 2018-01-14 650 0

## CodeSmith 创建Ado.Net自定义模版（二）

数据库 sql c#

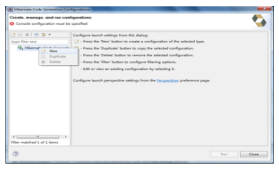
CodeSmith 创建Ado.Net自定义模版（二） 接第一篇： CodeSmith 创建Ado.Net自定义模版（一） 建立第二个C# Template: Step2_Model.cst（实体类模版） 代码： <%@ CodeTemplate Language="C#" TargetLanguage="C#"

Porschev 2018-01-16 449 0

## eclipse从数据库逆向生成Hibernate实体类

其他

做项目必然要先进行数据库表设计，然后根据数据库设计建立实体类(VO)，这是理所当然的，但是到公司里做项目后，让我认识到，没有说既进行完数据库设计后还要再“自己”建立一变VO。意思是，



似水的流年 2018-01-18 754 0

## 使用View Model从表现层分离领域模型

android

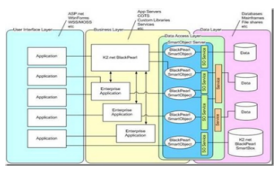
Model–View–Controller(模型–视图–控制器，MVC) 模式将你的软件组织并分解成三个截然不同的角色： Model 封装了你的应用数据、应用流程和业务逻辑。 View 从 Model 获取数据并格式化数据以进行显示。 Controller 控制程序流程，接收输入，并把它

张善友 2018-01-19 996 0

## BlackPearl 的 ServiceObject 开发部署

.net 存储

K2“BlackPearl”提供一个平台，用于管理和利用现有的跨越任何数量的业务系统的业务信息，以降低成本。它考虑到决策的制定，线路的排定以及报告，并根据360度的可见性进入到业务实体中，譬如客

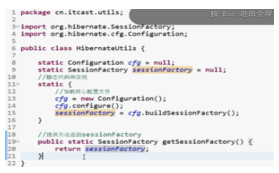


张善友 2018-01-19 370 0

## hibernate–笔记

其他

什么是 hibernate 框架 1.hibernate 框架应用在 javaee 三次结构中 dao 层框架 2.在 dao 层里面对数据库做curd 操作， 使用hibernate 做crud 操作, hibernate 底层代码就



用户1197315 2018-01-22 528 0

## ORM中的继承关系映射全解——单表继承体系、一实体一具体表、一实体一扩展表、接口映射

数据处理

实体继承是基于OO和关系型数据库软件系统设计中的一个重要主题。本文通过基于NBear的实例解析ORM中的实体继承体系映射的方方面面。 本文涉及的内容包括： 1. 单表继承体系 2. 一实体一具体表 3. 一实体一扩展表 4. 接口实现映射vs基类继承映射 1.

菩提树下的杨过 2018-01-22 1.8K 0

## EF基础知识小记一

其他

1、EF等ORM解决方案出现的原因 因为软件开发中分析和解决问题的方法已经接近成熟,然后关系型数据库却没有,很多年来,数据依然是保存在表行列这样的模式里,所以,在面相对象和高度标准化的数据库中产生了一个失配(不匹配、阻抗失配， 微软的安德斯.海尔

郑小超. 2018-01-26 844 0

## Linq基础知识小记四之操作EF

其他

1、EF简介 EF之于Linq,EF是一种包含Linq功能对象关系映射技术.EF对数据库架构和我们查询的类型进行更好的解耦,使用EF,我们查询的对象不再是C#类,而是更高层的抽象:Entity Data Model,这提供了额外的灵活性,但在性能和简单性上面也会有所损失. EF的

郑小超. 2018-01-26 826 0

---

## EF基础知识小记二

asp.net

1、EF的常用使用场景 (1)、维护一个已经存在的数据库,VS提供了工具帮助我们把数据库中的表和视图等对象导入到实体框架.[数据库=>模型(Database First)] (2)、通过VS提供的实体设计器设计表模型,然后从头开始添加实体类型、类型间的关联以及继承

郑小超. 2018-01-26 735 0

---

## 福利到！Rafy(原OEA)领域实体框架 2.22.2067 发布！

数据库 sql 分布式 http

距离“上次框架完整发布”已经过去了一年半了，应群中的朋友要求，决定在国庆放假之际，把最新的框架发布出来，并把帮助文档整理出来，这样可以方便大家快速上手。 08104816-68bbd9d568c049c08150b6cc83d1ac15.gif 发布内容 注意，本次发布，只

用户1172223 2018-01-26 966 0

---

## 一步一步创建ASP.NET MVC5程序[Repository+Autofac+Automapper+SqlSugar](三)

其他

上一篇《一步一步创建ASP.NET MVC5程序[Repository+Autofac+Automapper+SqlSugar](二)》我们通过如下操作：

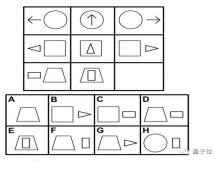
Rector 2018-01-30 933 0

---

## 击败水哥还不算最强，有个人工智能已经通过人类的智商测试了

机器学习 人工智能 大数据

这几天百度的人工智能在《最强大脑》击败“水哥”王昱珩，再次引发了人们对于人工智能的热议。然而人工智能在人脸识别的比赛中击败水哥，还远远称不上“最强大脑”，因为模式识别只是人工智能的



量子位 2018-01-30 588 0

加载更多

Copyright © 2013 – 2023 Tencent Cloud. All Rights Reserved. 腾讯云 版权所有
深圳市腾讯计算机系统有限公司 ICP备案/许可证号：粤B2-20090059 深公网安备号 44030502008569
腾讯云计算（北京）有限责任公司 京ICP证150476号 | 京ICP备11018762号 | 京公网安备号11010802020287